

UNIVERSITÉ DE LORRAINE
ENSEM — ÉCOLE NATIONALE SUPÉRIEURE D'ÉLECTRICITÉ ET DE
MÉCANIQUE

Rapport à mi-parcours

Année Universitaire 2025/2026

Reconnaissance de chiffres manuscrits et de visages par traitement d'images

Auteurs :

SLALI Zakariae
CHARAFI Amal
LUK Alson
ELABBOUDI Jaber
MOUALDI Rihab
ER-RIYACHY Mohammed Barae

Tuteur :

M. Didier WOLF

Table des matières

1	État de l'art	2
1.1	Reconnaissance de chiffres manuscrits	2
1.2	Détection de visages	2
1.3	Reconnaissance et identification de visages	3
1.4	Systèmes de gestion de présence automatisés	3
1.5	Comparaison des environnements de développement	3
2	Avancement du projet	4
2.1	Reconnaissance de chiffres manuscrits (MNIST)	4
2.1.1	Démonstration : reconnaissance de chiffres multiples	5
2.2	Reconnaissance faciale en temps réel (python)	6
2.2.1	Phase d'entraînement (Google Colab)	6
2.2.2	Phase de reconnaissance en temps réel	6
2.3	Application : système d'appel automatique	8
2.4	Comparaison Python vs MATLAB	8
2.4.1	Difficulté rencontrée : l'effet de la PCA	10
2.4.2	Reconnaissance faciale : Implémentation d'un CNN sous MATLAB	10
3	Planning et gestion du projet	12
	Bibliographie	13

1. État de l'art

Notre projet touche à deux grands problèmes de la vision par ordinateur : reconnaître des chiffres écrits à la main et identifier des visages en temps réel. Ces deux tâches ont une longue histoire en recherche, et les outils disponibles aujourd'hui permettent d'obtenir de très bons résultats même sur du matériel ordinaire. Voici un tour d'horizon des travaux et des méthodes qui ont guidé nos choix techniques.

1.1 Reconnaissance de chiffres manuscrits

Pour entraîner et tester nos modèles, nous utilisons le dataset MNIST (*Modified National Institute of Standards and Technology*), qui regroupe 70 000 images de chiffres manuscrits en niveaux de gris (60 000 pour l'entraînement et 10 000 pour le test). Publié par LeCun et al. en 1998 [1], ce jeu de données est devenu la référence standard pour comparer les algorithmes de classification d'images. Chaque image fait 28×28 pixels, centrée et normalisée, ce qui facilite les comparaisons entre méthodes [2].

Trois algorithmes ont retenu notre attention. Le k-NN (*k plus proches voisins*) est le plus simple : il classe une image en regardant ses voisins les plus proches dans l'espace des pixels. Pas d'entraînement au sens propre, mais lent à l'inférence sur de grands datasets. Le SVM, introduit par Cortes et Vapnik en 1995 [3], cherche à séparer les classes par un hyperplan à marge maximale, il donne de très bons résultats sur MNIST avec moins de 1 % d'erreur. Le CNN, popularisé par l'architecture LeNet-5 de LeCun et al. [1], exploite la structure spatiale des images grâce à des couches de convolution ; c'est aujourd'hui l'approche la plus performante, avec des taux d'erreur inférieurs à 0,3 % sur les modèles récents.

En pratique, nous nous appuyons sur scikit-learn [4] pour le k-NN et le SVM, et sur TensorFlow/Keras pour le CNN.

1.2 Détection de visages

Dans notre projet, deux méthodes de détection sont utilisées selon le contexte.

Pour la reconnaissance en temps réel (`livefa.py`), on utilise le classificateur **Haar-cascade** d'OpenCV : rapide, léger, il tourne sans GPU et détecte les visages dans chaque frame du flux webcam en quelques millisecondes.

Pour la phase d'entraînement, on préfère **MTCNN** (*Multi-task Cascaded Convolutional Networks*) [5], qui est nettement plus précis. MTCNN travaille en trois passes successives (P-Net, R-Net, O-Net) et localise en même temps le visage et ses points de repère faciaux (yeux, nez, coins de la bouche). Cette précision est nécessaire pour extraire des visages propres qui seront ensuite encodés par FaceNet. L'implémentation complète du pipeline est détaillée dans la vidéo de Abdirayimov (2024) [6], qui a servi de base directe à notre code.

1.3 Reconnaissance et identification de visages

Détecter un visage ne suffit pas, il faut encore l'identifier. Notre approche repose sur FaceNet [7], un réseau convolutif développé chez Google en 2015 qui projette chaque image de visage dans un espace de 128 dimensions. Dans cet espace, deux photos de la même personne donnent des vecteurs proches, et deux personnes différentes donnent des vecteurs éloignés. FaceNet atteint 99,63 % de précision sur le benchmark *Labeled Faces in the Wild*.

Concrètement, on utilise **keras-facenet**, une version pré-entraînée compatible TensorFlow. Une fois les embeddings calculés pour chaque personne de la base, on entraîne un SVM linéaire [3] pour classifier les identités. Un seuil de confiance à 0,7 permet de rejeter les visages non reconnus plutôt que de forcer une identification incorrecte.

1.4 Systèmes de gestion de présence automatisés

L'idée d'automatiser la prise de présence par reconnaissance faciale n'est pas nouvelle. Trivedi et al. (2022) ont développé un système similaire dans un contexte universitaire [8], et Lateef et Kamil (2023) ont comparé plusieurs pipelines sur ce même problème, leur conclusion est que MTCNN+FaceNet donne les meilleurs résultats [9]. Une revue systématique de Budiman et al. (2023) confirme la supériorité des approches CNN sur les méthodes classiques comme LBPH pour ce type d'application [10]. Ces travaux nous confortent dans nos choix techniques.

1.5 Comparaison des environnements de développement

Python domine aujourd'hui le domaine de l'IA grâce à des bibliothèques comme scikit-learn [4], TensorFlow, OpenCV et **keras-facenet**. MATLAB reste intéressant pour le prototypage rapide et la visualisation, avec ses toolboxes intégrées pour le traitement d'images et le deep learning. Comparer les deux sur les mêmes tâches est l'un des fils conducteurs de notre projet.

2. Avancement du projet

2.1 Reconnaissance de chiffres manuscrits (MNIST)

On a implémenté trois algorithmes sur MNIST [2] en Python, puis on les a comparés sur les mêmes données :

- **k-NN** ($k = 3$) : aucun entraînement à proprement parler, mais très lent à l'inférence quand le dataset est grand.
- **SVM (noyau RBF)** [3] : entraîné via scikit-learn [4], bon compromis précision/vitesse.
- **CNN** [1] : deux couches de convolution sous TensorFlow/Keras, meilleure précision des trois mais entraînement plus long.

Les résultats sont résumés dans le tableau 2.1. Le CNN prend la tête avec 98,70 % de précision, mais il lui faut 203 secondes pour s'entraîner. Le k-NN est quasi instantané à entraîner mais met 1,6 secondes pour classer 2000 images, trop lent pour une application temps réel. Le SVM est le meilleur compromis : 95 % de précision et seulement 6,9 secondes d'inférence.

Algorithme	Précision	Temps d'entraînement	Temps d'inférence
k-NN ($k = 3$)	92,35 %	0,01 s	1,6 s
SVM (RBF)	95,40 %	11,87 s	6,9 s
CNN	98,70 %	203,07 s	2,4 s

TABLE 2.1 – Comparaison des algorithmes sur MNIST (Python). k-NN et SVM évalués sur 10 000 images d'entraînement / 2 000 de test ; CNN sur 60 000 / 10 000.

Algorithme	Précision	T. train	T. test
k-NN (k=3)	92.35%	0.01s	1.16s
SVM (RBF)	95.40%	11.87s	6.93s
CNN	98.70%	203.07s	2.40s

Note : k-NN et SVM ont été évalués sur un sous-ensemble de 10000 images d'entraînement et 2000 images de test. Le CNN a été entraîné sur la totalité des 60 000 images.

FIGURE 2.1 – Résultats obtenus en Python sur MNIST : précision et temps d'exécution pour k-NN, SVM et CNN.

2.1.1 Démonstration : reconnaissance de chiffres multiples

Afin de valider le modèle CNN en conditions réelles, nous avons développé un script de détection et reconnaissance de plusieurs chiffres sur une même image. Le système détecte automatiquement chaque chiffre par analyse de contours, le recadre, le redimensionne en 28×28 pixels et le soumet au CNN. La figure 2.2 illustre le résultat sur une image contenant 10 chiffres manuscrits : 8 chiffres sur 10 sont correctement reconnus avec une confiance supérieure à 95 %. Deux échecs sont observés :

- Le chiffre **7** est confondu avec un 3 (confiance 60 %), en raison de son tracé penché qui s'apparente à un 3 une fois réduit en 28×28 pixels.
- Le chiffre **1** est prédit comme un 4 (confiance 48 %), car il se retrouve partiellement fusionné avec le 9 adjacent lors du recadrage automatique, perturbant la forme perçue par le CNN.

Ces erreurs mettent en évidence deux limites connues du pipeline : la sensibilité aux styles d'écriture non standards et la difficulté de segmentation lorsque deux chiffres sont proches. Des améliorations possibles incluent la data augmentation lors de l'entraînement (rotations, déformations) et un algorithme de séparation plus robuste des chiffres adjacents.

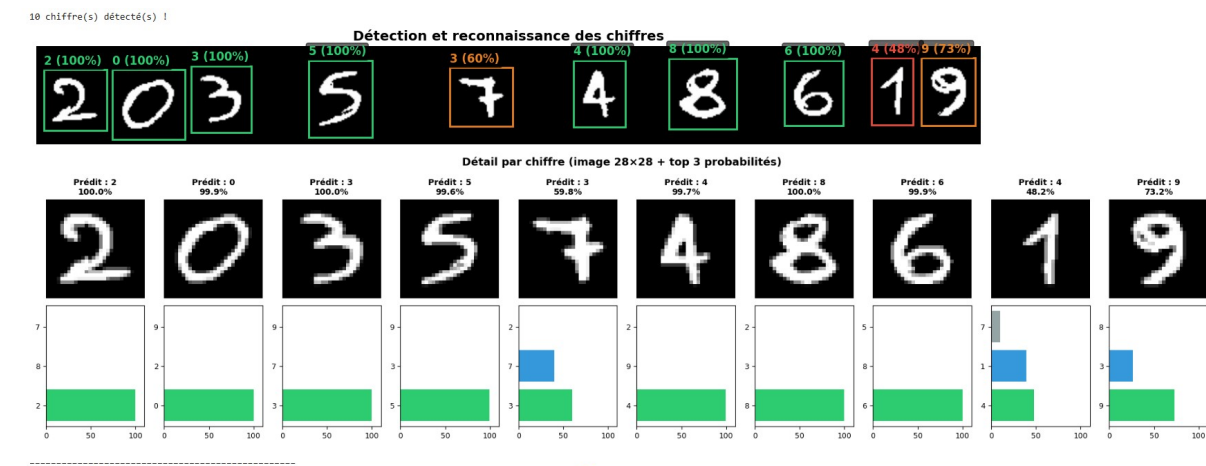


FIGURE 2.2 – Démonstration de la reconnaissance de 10 chiffres manuscrits par le CNN. 8/10 chiffres correctement identifiés. Le chiffre 7 est confondu avec un 3 (60 %) et le chiffre 1 est confondu avec un 4 (48 %) en raison de sa proximité avec le 9 adjacent.

2.2 Reconnaissance faciale en temps réel (python)

Le pipeline de reconnaissance faciale a été développé entièrement en Python, en suivant l'approche présentée par Abdirayimov (2024) [6].

2.2.1 Phase d'entraînement (Google Colab)

On commence par constituer une base de données de photos pour chaque personne à reconnaître. MTCNN [5] détecte et extrait le visage dans chaque photo, qu'on redimensionne à 160×160 pixels. FaceNet [7] convertit ensuite chaque visage en un vecteur de 128 valeurs via `keras-facenet`. Un SVM [3] est enfin entraîné sur ces vecteurs via `scikit-learn` [4] pour apprendre à distinguer les identités. Le modèle est sauvegardé en `pickle` pour être rechargé directement en temps réel.

2.2.2 Phase de reconnaissance en temps réel

En conditions réelles, la webcam capture le flux vidéo en continu. Haarcascade détecte les visages dans chaque frame, FaceNet [7] calcule leur embedding, et le SVM prédit l'identité. Si la confiance dépasse 0,7, la présence est enregistrée dans un fichier CSV avec le nom, la date et l'heure. Un délai anti-doublon de 60 secondes évite d'enregistrer la même personne plusieurs fois de suite.

Les figures 2.3 et 2.4 montrent les deux premières étapes du pipeline : MTCNN repère le visage dans la photo originale (rectangle bleu), puis on découpe et redimensionne ce visage en 160×160 pixels avant de l'envoyer à FaceNet. La figure 2.5 donne un aperçu de la base de données complète après extraction, et la figure 2.6 visualise un vecteur d'embedding (les 512 valeurs qui résument l'identité faciale d'une personne).

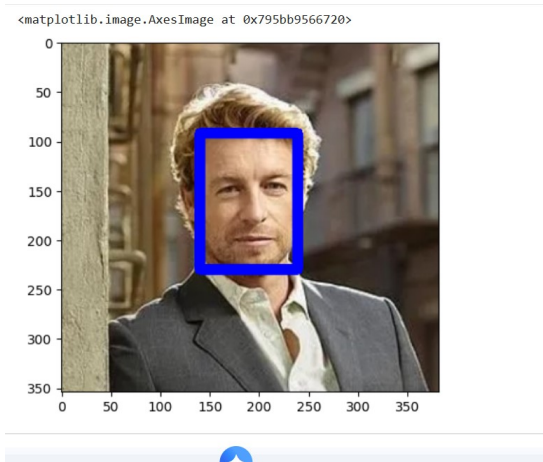


FIGURE 2.3 – Détection du visage par MTCNN (rectangle bleu).

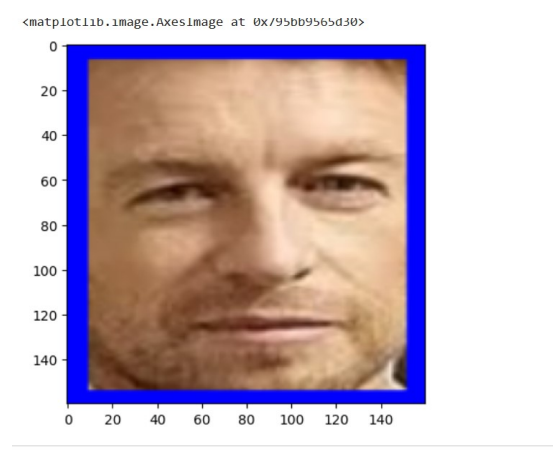
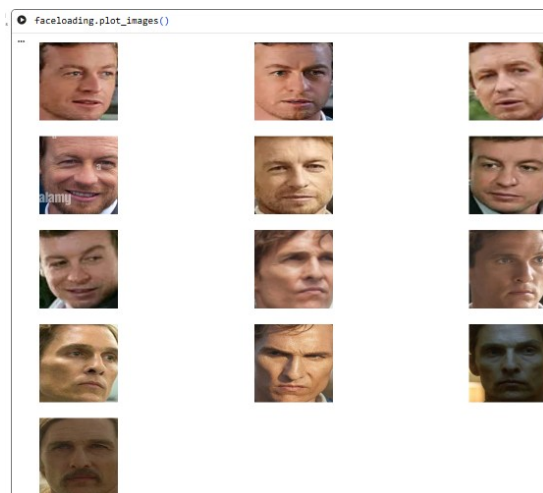
FIGURE 2.4 – Visage extrait et redimensionné en 160×160 px.

FIGURE 2.5 – Base de données de visages chargée via FACELOADING.

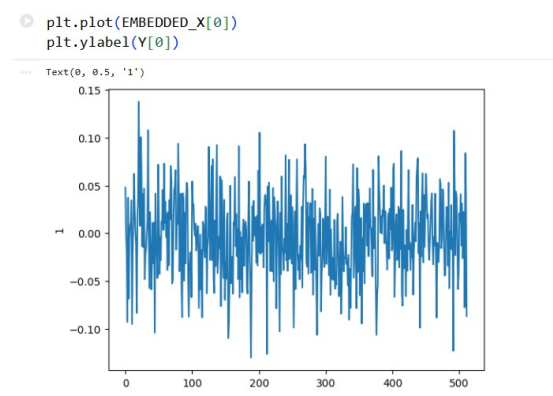


FIGURE 2.6 – Vecteur d'embedding FaceNet du premier visage (512 dimensions).

Le tableau 2.2 présente les taux de précision obtenus après entraînement du classificateur SVM sur les embeddings FaceNet.

Modèle	Précision entraînement	Précision test
SVM linéaire + FaceNet (Python)	100 %	100 %

TABLE 2.2 – Taux de précision du classificateur SVM entraîné sur les embeddings FaceNet.

Le SVM atteint **100 %** de précision sur les deux ensembles. Ce score s'explique par la qualité des embeddings FaceNet, qui séparent très nettement les identités dans l'espace vectoriel. Il faut cependant relativiser : la base de données est petite, et les conditions

d'entraînement sont idéales. Les tests en conditions réelles (éclairage variable, angles différents) seront le vrai banc d'essai du système.

2.3 Application : système d'appel automatique

L'application concrète du projet est un système d'appel automatique en amphithéâtre : une caméra à l'entrée identifie les étudiants à leur passage et génère la liste des présents en fin de cours, sans émargement papier. Des systèmes similaires ont été testés dans d'autres universités [8, 9], ce qui valide la faisabilité de l'approche.

2.4 Comparaison Python vs MATLAB

Sur MNIST, on a implémenté k-NN et SVM dans les deux environnements et comparé les résultats. Le tableau 2.3 résume les taux de reconnaissance obtenus.

Algorithme	Python	MATLAB
k-NN ($k = 3$)	92,35 %	92,00 %
SVM	95,40 %	90,90 %
CNN	98,70 %	—

TABLE 2.3 – Comparaison des taux de reconnaissance sur MNIST entre Python et MATLAB.

Le k-NN donne des résultats quasi identiques dans les deux environnements (92,35 % en Python, 92 % en MATLAB), ce qui valide la cohérence des implémentations. Le SVM est légèrement meilleur en Python (95,40 % contre 90,90 %) grâce au noyau RBF mieux paramétré. Le CNN n'a pas encore été implémenté en MATLAB.

Les figures 2.7 et 2.9 montrent la matrice de confusion et les exemples de mauvaises classifications obtenus avec le SVM MATLAB. La figure 2.8 illustre un exemple de bonne classification (7 prédit comme 7).

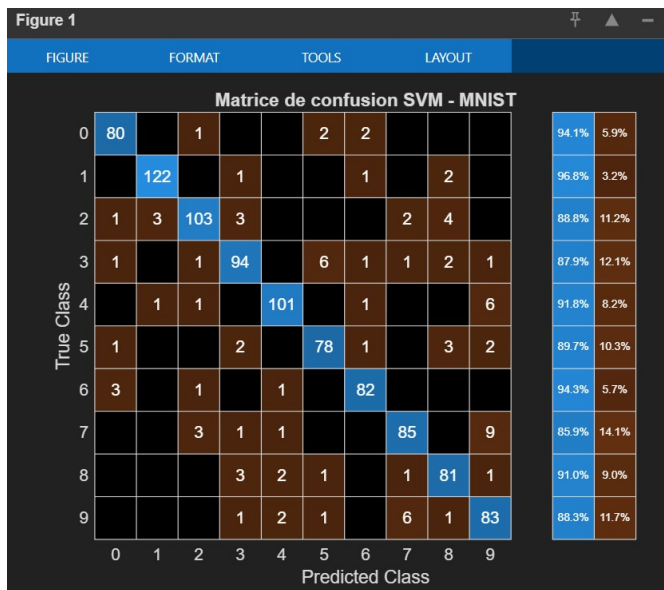


FIGURE 2.7 – Matrice de confusion SVM MATLAB sur MNIST (90,90 %).

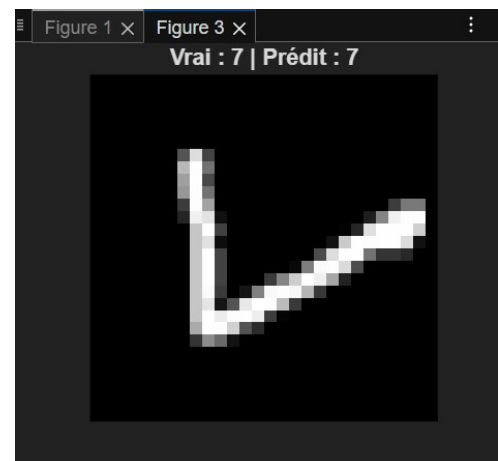


FIGURE 2.8 – Exemple correct : Vrai 7, Prédit 7.

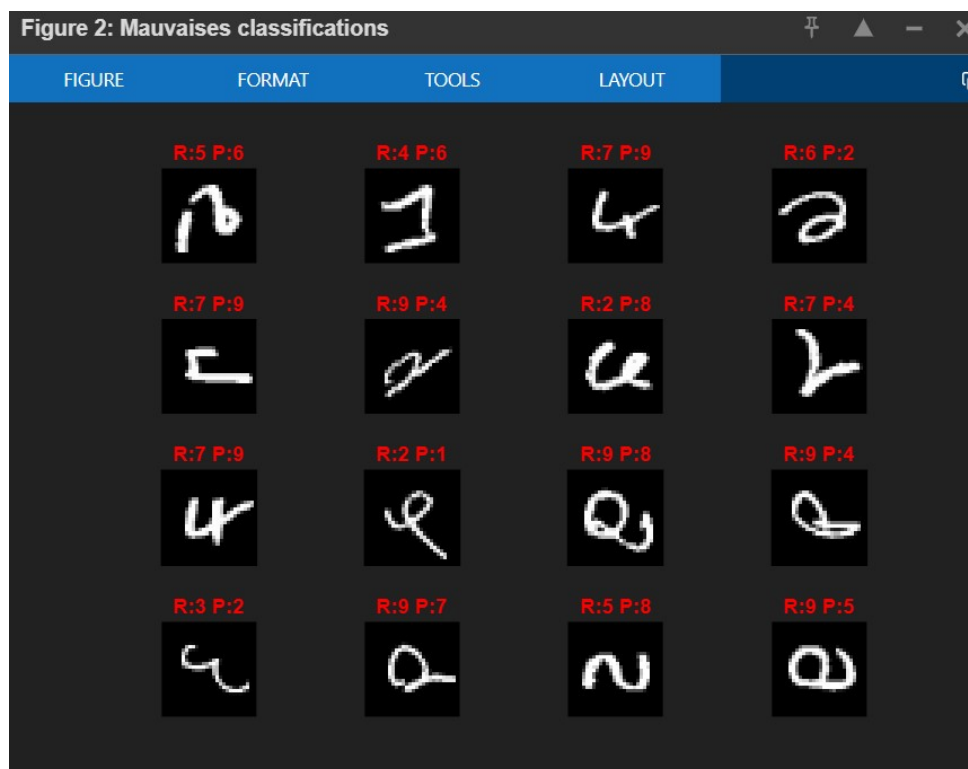


FIGURE 2.9 – Exemples de mauvaises classifications du SVM MATLAB. Les chiffres cursifs et ambigus (7 confondu avec 9, 2 confondu avec 1) représentent les cas d'échec les plus fréquents.

2.4.1 Difficulté rencontrée : l'effet de la PCA

Lors du développement MATLAB, on a d'abord appliqué une réduction de dimension par PCA (100 composantes) avant la classification SVM, dans le but d'accélérer l'entraînement. Cette décision a conduit à un taux de reconnaissance de seulement 12,60 %, le modèle prédisant systématiquement la classe 1 quelle que soit l'entrée. L'analyse a montré que la projection PCA, appliquée sur un sous-échantillon de 10 000 images seulement, avait détruit l'information discriminante nécessaire au SVM. En supprimant la PCA et en travaillant directement sur les 784 pixels normalisés, le taux est remonté à 90,90 %. Cette expérience illustre un piège classique : une réduction de dimension mal calibrée dégrade les performances bien plus qu'elle ne les améliore.

2.4.2 Reconnaissance faciale : Implémentation d'un CNN sous MATLAB

Afin de pousser la comparaison entre les deux environnements, nous avons conçu un pipeline complet de reconnaissance faciale sous MATLAB, allant de la détection à l'entraînement d'un réseau de neurones convolutif (CNN) personnalisé. Cette approche fait écho au pipeline Python (MTCNN + FaceNet) mais s'appuie ici sur les outils natifs de MATLAB.

Prétraitement et augmentation des données : La première étape consiste à repérer et recadrer les visages sur les images brutes en utilisant l'algorithme de Viola-Jones via l'objet `vision.CascadeObjectDetector()`. Ces visages sont ensuite convertis en RGB et redimensionnés au format 100×100 pixels. Pour enrichir notre base d'entraînement et rendre le modèle plus robuste aux inclinaisons de la tête, nous avons intégré une étape d'augmentation de données par des rotations aléatoires comprises entre -15° et $+15^\circ$.

Entraînement et résultats : L'entraînement a été réalisé avec l'optimiseur SGDM (taux d'apprentissage initial de 0,01) par lots de 16 images sur 30 époques. La figure 2.10 montre une convergence rapide et stable : le modèle a atteint une précision de 100 % sur les données d'entraînement en un peu plus de 6 minutes d'exécution sur processeur (CPU).

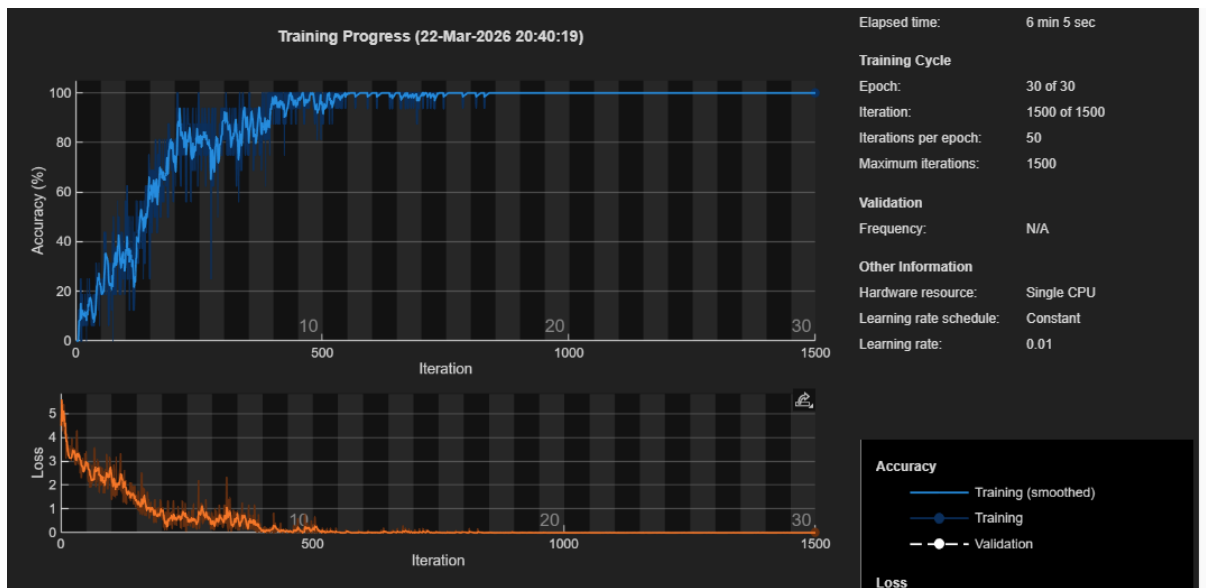


FIGURE 2.10 – Courbe d'apprentissage du CNN sous MATLAB (précision et perte). Atteinte de 100 % de précision en 30 époques.

Phase d'inférence : Pour valider l'apprentissage, un script de prédiction permet de traiter de nouvelles entrées. Il extrait automatiquement le visage, le normalise et le soumet au réseau. La figure 2.11 confirme que le modèle parvient à classifier correctement de nouvelles images en les associant au bon label (par exemple, le sujet « s1 »), validant ainsi l'architecture choisie sous MATLAB.



FIGURE 2.11 – Test d'inférence sous MATLAB : détection et reconnaissance réussies pour le sujet « s1 ».

3. Planning et gestion du projet

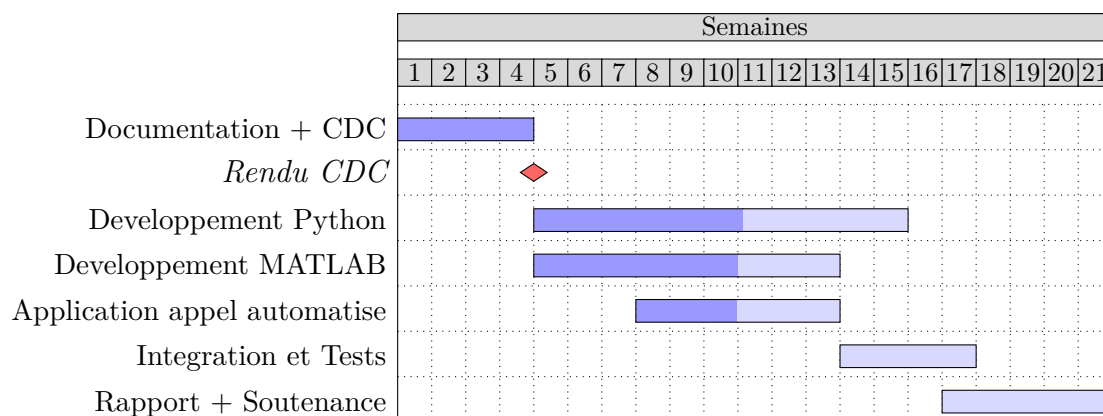


FIGURE 3.1 – Planning previsionnel – semaine 11

Bibliographie

- [1] Yann LECUN et al. “Gradient-based learning applied to document recognition”. In : *Proceedings of the IEEE* 86.11 (nov. 1998). Consulté le 15 février 2026, p. 2278-2324. URL : <https://hal.science/hal-03926082v1/document>.
- [2] Yann LECUN et Corinna CORTES. *MNIST handwritten digit database*. En ligne. Consulté le 01 janvier 2026. 1998. URL : <https://github.com/cvdfoundation/mnist>.
- [3] Corinna CORTES et Vladimir VAPNIK. “Support-Vector Networks”. In : *Machine Learning* 20.3 (1995). Consulté le 13 février 2026, p. 273-297. URL : https://www.marenglenbiba.net/dm/cortes_vapnik95.pdf.
- [4] Fabian PEDREGOSA et al. “Scikit-learn : Machine Learning in Python”. In : *Journal of Machine Learning Research* 12 (2011). Consulté le 14 mars 2026, p. 2825-2830. URL : <http://www.jmlr.org/papers/volume12/pedregosa11a/pedregosa11a.pdf>.
- [5] Kaipeng ZHANG et al. “Joint Face Detection and Alignment Using Multitask Cascaded Convolutional Networks”. In : *IEEE Signal Processing Letters* 23.10 (oct. 2016). Consulté le 13 décembre 2025, p. 1499-1503. URL : <https://arxiv.org/pdf/1604.02878>.
- [6] Sardor ABDIRAYIMOV. *Face Recognition in Python / Real-time / Part 2 / FaceNet, MTCNN, SVM*. Vidéo en ligne. Consulté le 01 février 2026. 2024. URL : <https://youtu.be/9niSMx2vxB4>.
- [7] Florian SCHROFF, Dmitry KALENICHENKO et James PHILBIN. “FaceNet : A Unified Embedding for Face Recognition and Clustering”. In : *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. Consulté le 14 mars 2026. Boston, juin 2015, p. 815-823. URL : <https://arxiv.org/pdf/1503.03832>.
- [8] Aparna TRIVEDI et al. “Face Recognition Based Automated Attendance Management System”. In : *International Journal of Scientific Research in Science and Technology* 9.1 (février 2022). Consulté le 01 mars 2026, p. 261-268. URL : <https://ijsrst.com/paper/9242.pdf>.

- [9] Ahmad S. LATEEF et Mohammed Y. KAMIL. “Facial Recognition Technology-Based Attendance Management System Application in Smart Classroom”. In : *Iraqi Journal for Computer Science and Mathematics* 4.3 (2023). Consulté le 04 mars 2026. URL : <https://ijcsm.researchcommons.org/cgi/viewcontent.cgi?article=1097&context=ijcsm>.
- [10] Andre BUDIMAN et al. “Student attendance with face recognition (LBPH or CNN) : Systematic literature review”. In : *Procedia Computer Science* 216 (2023). Consulté le 14 mars 2026, p. 31-38. URL : <https://www.sciencedirect.com/science/article/pii/S187705092202186X>.